

CO3 - AT1

Policy Optimization Assessment

ASSESSMENT - 1

**Coding Challenge – Vanilla Policy Gradient & REINFORCE
(Apply – L3)**

Name: P. Hithaishi

Reg No: 192425196

Course Code: MLA0305

CODE

```
!pip install gymnasium torch matplotlib

import gymnasium as gym

import torch

import torch.nn as nn

import torch.optim as optim

import numpy as np

import matplotlib.pyplot as plt

env = gym.make("CartPole-v1")

state_size = env.observation_space.shape[0]

action_size = env.action_space.n

class PolicyNetwork(nn.Module):

    def __init__(self):

        super().__init__()

        self.network = nn.Sequential(

            nn.Linear(state_size, 128),

            nn.ReLU(),

            nn.Linear(128, action_size),

            nn.Softmax(dim=-1)

        )

    def forward(self, state):

        return self.network(state)

policy = PolicyNetwork()

optimizer = optim.Adam(

    policy.parameters(),

    lr=0.01

)

gamma = 0.99

def select_action(state):
```

```

state = torch.FloatTensor(state)
probabilities = policy(state)
distribution = torch.distributions.Categorical(
    probabilities
)
action = distribution.sample()
log_probability = distribution.log_prob(action)
return action.item(), log_probability

def calculate_returns(rewards):
    returns = []
    G = 0
    for reward in reversed(rewards):
        G = reward + gamma * G
        returns.insert(0, G)
    returns = torch.tensor(
        returns,
        dtype=torch.float32
    )
    returns = (
        returns - returns.mean()
    ) / (returns.std() + 1e-8)
    return returns

def evaluate_policy(epochs=20):
    scores = []

    for _ in range(epochs):
        state, info = env.reset()
        total_reward = 0
        done = False

```

```

while not done:
    state_tensor = torch.FloatTensor(state)
    probabilities = policy(
        state_tensor
    )
    action = torch.argmax(
        probabilities
    ).item()
    state, reward, terminated, truncated, info = env.step(action)
    done = terminated or truncated
    total_reward += reward
    scores.append(total_reward)
return np.mean(scores)
before_training = evaluate_policy()
print(
    "Average Reward Before Training:",
    round(before_training, 2)
)
episodes = 500
training_rewards = []
for episode in range(episodes):
    state, info = env.reset()
    log_probabilities = []
    rewards = []

    done = False
    while not done:
        action, log_probability = select_action(state)
        next_state, reward, terminated, truncated, info = env.step(action)

```

```

done = terminated or truncated

log_probabilities.append(
    log_probability
)

rewards.append(reward)

state = next_state

returns = calculate_returns(rewards)

loss = 0

for log_probability, G in zip(
    log_probabilities,
    returns
):
    loss += -log_probability * G

optimizer.zero_grad()

loss.backward()

optimizer.step()

total_reward = sum(rewards)

training_rewards.append(
    total_reward
)

if (episode + 1) % 50 == 0:

    average = np.mean(
        training_rewards[-50:]
    )

    print(
        f"Episode {episode+1}: "
        f"Average Reward = {average:.2f}"
    )

```

```
after_training = evaluate_policy()

print(
    "\nAverage Reward After Training:",
    round(after_training, 2)
)

plt.figure(figsize=(10, 5))
plt.plot(
    training_rewards,
    alpha=0.5,
    label="Episode Reward"
)

window = 20
moving_average = np.convolve(
    training_rewards,
    np.ones(window) / window,
    mode="valid"
)

plt.plot(
    range(window - 1, len(training_rewards)),
    moving_average,
    linewidth=2,
    label="Moving Average"
)

plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title(
    "REINFORCE Training Performance"
)
```

```
plt.legend()
plt.grid()
plt.show()
labels = [
    "Before Training",
    "After Training"
]
values = [
    before_training,
    after_training
]
plt.figure(figsize=(8, 5))
bars = plt.bar(
    labels,
    values
)
plt.ylabel("Average Reward")
plt.title(
    "REINFORCE Performance Comparison"
)
plt.grid(
    axis="y",
    alpha=0.3
)

for bar in bars:
    height = bar.get_height()

    plt.text(
```

```
        bar.get_x() + bar.get_width() / 2,  
        height,  
        f'{height:.2f}',  
        ha="center",  
        va="bottom"  
    )  
plt.show()  
env.close()
```

OUTPUT

Average Reward Before Training: 9.35
Episode 50: Average Reward = 62.36
Episode 100: Average Reward = 366.84
Episode 150: Average Reward = 372.18
Episode 200: Average Reward = 372.84
Episode 250: Average Reward = 117.38
Episode 300: Average Reward = 107.14
Episode 350: Average Reward = 151.58
Episode 400: Average Reward = 401.16
Episode 450: Average Reward = 236.84
Episode 500: Average Reward = 490.38
Average Reward After Training: 500.0

